OSY Practical Examination

Simple Answers and Python Programs — Questions 1 to 13

Use these commands/programs as a study guide. Linux distributions may differ slightly; scheduling examples assume lower priority number means higher priority.

1. Basic Linux Commands

1(a) Record all activities performed in terminal

```
script record.txt
# Perform commands/activities
exit
```

The terminal session is saved in record.txt.

1(b) Display calendar for April 2024

```
cal 4 2024
```

1(c) Display present working directory

```
pwd
```

1(d) Acquire superuser status

```
sudo su
```

1(e) Create TYAN directory and set permissions using octal method

```
mkdir TYAN
chmod 766 TYAN
```

766 gives owner read/write/execute, group read/write, and others read/write. Since a directory needs execute permission to be entered/accessed, use 777 if group and others must also be able to access it. For read + write + execute for everyone: chmod 777 TYAN.

1(f) Display all attributes/details of TYAN directory

```
ls -ld TYAN
```

2. Linux Commands

2(a) Display birth-month calendar and find weekday

```
cal 4 2008
```

29 April 2008 was Tuesday.

2(b) Display previous, current and next month calendars

```
cal -3
```

On systems where cal -3 is unsupported, use: cal -3 (util-linux cal).

2(c) Display full weekday name

```
date +%A
```

2(d) List currently logged-in users

```
who
```

2(e) Display terminal name

```
tty
```

2(f) Display PID and PPID of all processes

```
ps -ef
```

The PID and PPID columns show process ID and parent process ID.

3. Date, Help and Process Commands

3(i) Display login shell

```
echo $SHELL
```

3(ii) Display help for wc command

```
wc --help
```

3(iii) Date formats

```
date +%y       # two-digit year
date +%B       # full month name
date +%d/%m/%Y # dd/mm/yyyy
date +%H:%M:%S # current time, 24-hour format
```

3(iv) Display PID and PPID of all processes

```
ps -ef
```

3(v) List currently logged-in users

```
who
```

4. File Handling Commands

4(a) Create small.txt with five lines; append short.txt to long.txt ten times

```
cat > small.txt
# Type 5 lines, then press Ctrl+D

cat > short.txt
# Type 5 lines, then press Ctrl+D

for i in {1..10}; do cat short.txt >> long.txt; done
```

If short.txt contains 5 lines and long.txt is empty before the loop, long.txt will contain 50 lines. If the task literally requires appending short.txt to long.txt, ensure short.txt exists first.

4(b) Display first 10 lines

```
head -10 long.txt
```

4(c) Count lines

```
wc -l long.txt
```

4(d) Create ABCD.txt and copy it as XYZ.txt

```
touch ABCD.txt
cp ABCD.txt XYZ.txt
```

4(e) Rename ABCD.txt to AAA.txt and delete XYZ.txt

```
mv ABCD.txt AAA.txt
rm XYZ.txt
```

4(f) Create KKWP with 10 lines and split into 5 files tagged Nashik

```
cat > KKWP
# Type 10 lines, then press Ctrl+D
split -n 5 KKWP Nashik
```

The output files will have names beginning with Nashik (for example, Nashikaa, Nashikab, etc.).

5. File and Directory Commands

5(a) Create Unit1 and Unit2 using cat

```
cat > Unit1
# Type content, then Ctrl+D
cat > Unit2
# Type content, then Ctrl+D
```

5(b) Copy contents of Unit1 as Unit3

```
cp Unit1 Unit3
```

5(c) Rename Unit1 to Lesson1

```
mv Unit1 Lesson1
```

5(d) Create KKWP directory and display its default permissions

```
mkdir KKWP
ls -ld KKWP
```

The displayed permissions depend on the system's umask.

5(e) Create AN.txt inside KKWP and list files in KKWP

```
touch KKWP/AN.txt
ls KKWP
```

5(f) Create KKWP file with 10 lines and split into 5 files tagged Nashik

```
cat > KKWP_file
# Type 10 lines, then Ctrl+D
split -n 5 KKWP_file Nashik
```

A file and a directory cannot share the same name in the same location. This uses KKWP_file to avoid conflict with the KKWP directory created in part (d).

6. Run Level, Messaging and Terminal Permissions

6(a) Display current run level using who

```
who -r
```

6(b) Display current run level

```
who -r
```

6(c) Append contents of one file to another using cat

```
cat source.txt >> destination.txt
```

6(d) Send a message from a file using wall

```
wall < message.txt
```

6(e) Check terminal message permission status

```
mesg
```

mesg y allows messages; mesg n disables them. Use mesg y or mesg n to change the status.

7. FCFS CPU Scheduling

7(a) Python program to calculate average waiting and turnaround time using FCFS

```
n = int(input("Number of processes: "))
at, bt = [], []
for i in range(n):
    at.append(int(input(f"Arrival time P{i}: ")))
    bt.append(int(input(f"Burst time P{i}: ")))

order = sorted(range(n), key=lambda i: (at[i], i))
ct = [0] * n
time = 0
for i in order:
    time = max(time, at[i])
    time += bt[i]
    ct[i] = time

wt, tat = [], []
for i in range(n):
    tat.append(ct[i] - at[i])
    wt.append(tat[i] - bt[i])

for i in range(n):
    print(f"P{i}: WT={wt[i]}, TAT={tat[i]}")
print("Average waiting time =", sum(wt) / n)
```

```
print("Average turnaround time =", sum(tat) / n)
```

7(b) Solve FCFS for given processes

Process	Arrival	Burst	Completion	Turnaround	Waiting
P0	0	5	5	5	0
P1	1	3	8	7	4
P2	2	8	16	14	6
P3	3	6	22	19	13

Average waiting time = (0 + 4 + 6 + 13) / 4 = 5.75 ms. Average turnaround time = (5 + 7 + 14 + 19) / 4 = 11.25 ms.

Gantt chart: 0 | P0 | 5 | P1 | 8 | P2 | 16 | P3 | 22

8. SJF (Non-preemptive) CPU Scheduling

8(a) Python program for SJF non-preemptive

```
n = int(input("Number of processes: "))
p = []
for i in range(n):
    at = int(input(f"Arrival time P{i+1}: "))
    bt = int(input(f"Burst time P{i+1}: "))
    p.append([f"P{i+1}", at, bt])

time = 0
done = []
while len(done) < n:
    ready = [x for x in p if x[0] not in [d[0] for d in done] and x[1] <= time]
    if not ready:
        time = min(x[1] for x in p if x[0] not in [d[0] for d in done])
        continue
    x = min(ready, key=lambda a: a[2])
    start = time
    time += x[2]
    ct = time
    tat = ct - x[1]
    wt = tat - x[2]
    done.append([x[0], start, ct, tat, wt])

for d in done:
    print(d[0], "Start", d[1], "CT", d[2], "TAT", d[3], "WT", d[4])
print("Average WT =", sum(d[4] for d in done)/n)
print("Average TAT =", sum(d[3] for d in done)/n)
```

8(b) Solve given SJF non-preemptive problem

Process	Burst	Arrival	Start	Completion	Turnaround	Waiting
P1	6	0	0	6	6	0
P2	3	1	6	9	8	5
P3	4	2	9	13	11	7
P4	10	3	13	23	20	10

At time 0, only P1 has arrived, so it runs first. Then P2, P3 and P4 are selected by shortest burst time.

Average waiting time = (0 + 5 + 7 + 10) / 4 = 5.5 ms. Average turnaround time = (6 + 8 + 11 + 20) / 4 = 11.25 ms.

9. Priority CPU Scheduling (Preemptive)

9(a) Python program for priority preemptive scheduling (lower number = higher priority)

```
n = int(input("Number of processes: "))
p = []
for i in range(n):
    at = int(input(f"Arrival time P{i+1}: "))
    bt = int(input(f"Burst time P{i+1}: "))
```

```
    pr = int(input(f"Priority P{i+1} (lower = higher): "))
    p.append([f"P{i+1}", at, bt, pr, bt])

time = 0
ct = {}
while len(ct) < n:
    ready = [x for x in p if x[1] <= time and x[4] > 0]
    if not ready:
        time += 1
        continue
    x = min(ready, key=lambda a: (a[3], a[1]))
    x[4] -= 1
    time += 1
    if x[4] == 0:
        ct[x[0]] = time

wt, tat = [], []
for x in p:
    t = ct[x[0]] - x[1]
    w = t - x[2]
    print(x[0], "WT =", w, "TAT =", t)
    wt.append(w); tat.append(t)
print("Average WT =", sum(wt)/n)
print("Average TAT =", sum(tat)/n)
```

9(b) Solve the given priority preemptive problem

Assumption: smaller priority number means higher priority. At time 0 P1 starts; at time 2 P3 arrives with priority 1 and preempts P1.

Gantt chart: 0–2 P1 | 2–7 P3 | 7–11 P1 | 11–15 P2

Process	AT	BT	Priority	CT	TAT = CT–AT	WT = TAT–BT
P1	0	6	2	11	11	5
P2	1	4	3	15	14	10
P3	2	5	1	7	5	0

Average waiting time = (5 + 10 + 0) / 3 = 5 ms. Average turnaround time = (11 + 14 + 5) / 3 = 10 ms.

10. Round Robin CPU Scheduling

10(a) Python program for Round Robin

```
n = int(input("Number of processes: "))
bt = [int(input(f"Burst time P{i+1}: ")) for i in range(n)]
q = int(input("Time quantum: "))
rem = bt[:]
ct = [0] * n
time = 0
while any(x > 0 for x in rem):
    for i in range(n):
        if rem[i] > 0:
            run = min(q, rem[i])
            time += run
            rem[i] -= run
            if rem[i] == 0:
                ct[i] = time
tat = [ct[i] for i in range(n)]  # all arrivals are time 0
wt = [tat[i] - bt[i] for i in range(n)]
for i in range(n):
    print(f"P{i+1}: WT={wt[i]}, TAT={tat[i]}")
print("Average WT =", sum(wt)/n)
print("Average TAT =", sum(tat)/n)
```

10(b) Solve RR: quantum = 2 ms; all processes arrive at time 0

Gantt chart: 0 P1 2 P2 3 P3 5 P4 6 P5 8 P1 10 P5 12 P1 14 P5 15 P1 17 P1 19 P1 21 P1 23

Process	Burst	Completion	Turnaround	Waiting
P1	10	23	23	13

Process	Burst	Completion	Turnaround	Waiting
P2	1	3	3	2
P3	2	5	5	3
P4	1	6	6	5
P5	5	15	15	10

Average waiting time = (13 + 2 + 3 + 5 + 10) / 5 = 6.6 ms. Average turnaround time = (23 + 3 + 5 + 6 + 15) / 5 = 10.4 ms.

11. FIFO Page Replacement

11(a) Python program for FIFO page replacement

```
pages = list(map(int, input("Enter page reference string: ").split(",")))
capacity = int(input("Number of frames: "))
frames = []
faults = 0
pointer = 0
for page in pages:
    if page not in frames:
        faults += 1
        if len(frames) < capacity:
            frames.append(page)
        else:
            frames[pointer] = page
            pointer = (pointer + 1) % capacity
    print(page, ":", frames)
print("Page faults =", faults)
```

11(b) FIFO faults for 3 frames

Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

Number of page faults = 15.

12. LRU Page Replacement

12(a) Python program for LRU page replacement

```
pages = list(map(int, input("Enter page reference string: ").split(",")))
capacity = int(input("Number of frames: "))
frames = []
last_used = {}
faults = 0
for t, page in enumerate(pages):
    if page not in frames:
        faults += 1
        if len(frames) < capacity:
            frames.append(page)
        else:
            old = min(frames, key=lambda x: last_used[x])
            frames[frames.index(old)] = page
    last_used[page] = t
    print(page, ":", frames)
print("Page faults =", faults)
```

12(b) LRU faults for 3 frames

Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

Number of page faults = 12.

13. Optimal Page Replacement

13(a) Python program for Optimal page replacement

```
pages = list(map(int, input("Enter page reference string: ").split(",")))
capacity = int(input("Number of frames: "))
frames = []
faults = 0
```

```
for i, page in enumerate(pages):
    if page not in frames:
        faults += 1
        if len(frames) < capacity:
            frames.append(page)
        else:
            future = pages[i+1:]
            distances = []
            for f in frames:
                distances.append(future.index(f) if f in future else float("inf"))
            victim = frames[distances.index(max(distances))]
            frames[frames.index(victim)] = page
    print(page, ":", frames)
print("Page faults =", faults)
```

13(b) Optimal page faults for 3 frames

Reference string: 0,1,2,3,0,1,2,3,0,1,2,3,4,5,6,7

Number of page faults = 10.

Quick Revision: Final Numerical Answers

Question	Result
7(b) FCFS	Average WT = 5.75 ms; Average TAT = 11.25 ms
8(b) SJF non-preemptive	Average WT = 5.5 ms; Average TAT = 11.25 ms
9(b) Priority preemptive	Average WT = 5 ms; Average TAT = 10 ms
10(b) Round Robin, q=2	Average WT = 6.6 ms; Average TAT = 10.4 ms
11(b) FIFO, 3 frames	15 page faults
12(b) LRU, 3 frames	12 page faults
13(b) Optimal, 3 frames	10 page faults

Exam note: For priority scheduling, some textbooks use the opposite convention (larger number = higher priority). If your teacher specifies that convention, the priority answer will change.